claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_3e301d0b-e59\agent-a9c007634ac6c7c7f.jsonl`
- SHA-256: `9336904a0c64ef498053e7e7ea5752e61c45f1846b584f983e7ed0b072edcabe`
- Source modified: `2026-06-13T18:46:26+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_3e301d0b-e59`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T18:45:07.543Z)

DESIGN the person-feature computation as PURE functions (this is the math we must get right; it cannot be GPU-tested here).
Context: MULTI_SIGNAL_FUSION_PLAN.md §3.2/§4 (read it). Inputs available per shuttle-seeded window: per-frame person boxes with track ids (from PersonDetector ? read backend/pipeline/detectors/person_detector.py), per-frame shuttle (x,y,visible) from the WASB trajectory (read tracknet_runner.parse_trajectory_csv shape), an OPTIONAL court polygon (normalized 0-1) and a net line.
Define EXACT, scale/translation-INVARIANT formulas (counts/ratios/zone-membership ? never raw pixel coords) with edge cases (no shuttle visible, <2 persons, polygon None) for: players_in_court, two_sided_motion, mean_player_motion, in_court_ratio, shuttle_player_colocation. Also give point-in-polygon + net-split definitions. Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer (the indexer). Read the ACTUAL files and quote file:line. Be precise, not generic.

2. assistant (2026-06-13T18:45:08.990Z)

I'll start by reading the MULTI_SIGNAL_FUSION_PLAN.md to understand the context and requirements for the person-feature computation.

3. user (2026-06-13T18:45:09.155Z)

docs\MULTI_SIGNAL_FUSION_PLAN.md

4. assistant (2026-06-13T18:45:10.697Z)

Now let me read the MULTI_SIGNAL_FUSION_PLAN.md file to understand sections 3.2 and 4:

5. user (2026-06-13T18:45:11.075Z)

```
1       # Multi-signal rally fusion ? add a human (person) cue alongside the shuttle
2
3       **Status:** **P1 DONE** (2026-06-13) ? the person cue is extracted to
4       `backend/pipeline/detectors/person_detector.py` (`PersonDetector`), consumed by
`yolo_hybrid` with no
5       behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a
separate PR.
6       **Owner decisions captured** in [§9](#9-open-decisions-for-the-owner).
7
8       > **Companion docs:** the 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md);
9       > the modular-fusion architecture this builds on is **ADR-007**
([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md));
10      > the permissive-dependency rule is **ADR-002 / ADR-005**; the audio cue (the #1 next
```

```
cue, already
11      > scaffolded) is [archives/research/AUDIO_RALLY_DETECTION_PLAN.md](archives/research/AUD
IO_RALLY_DETECTION_PLAN.md).
12      > The vendor labels that gate the learned path come from
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)
13      > and the labeling spec in [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md).
14      > **How each cue here is added/tested without production regression** ? A/B, shadow
eval, and the
15      > promote-only-on-lift discipline ? is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)
(every P1?P4 cue
16      > below lands flag-gated, default OFF, promoted only on measured LOVO lift).
17
18      ---
19
20      ## 1. Context ? why this, why now
21
22      Rally detection today runs **one cue at a time**. Each segmenter independently turns a
single signal
23      into rally windows:
24
25      | Segmenter | Signal | File |
26      |---|---|---|
27      | `wasb_hybrid` / `tracknet_hybrid` | shuttle velocity (frozen WASB/TrackNet trajectory)
| `backend/pipeline/detectors/tracknet_runner.py` |
28      | `yolo_hybrid` (the current `default_segmenter`) | person velocity (torchvision +
ByteTrack) | `backend/pipeline/segmenters/yolo_hybrid.py` |
29      | `motion` | raw pixel motion | `backend/pipeline/segmenters/motion.py` |
30      | `gemini` | a cloud VLM watching the whole clip | `backend/providers/gemini.py` |
31
32      **There is no fusion.** The owner's directive: make the detector **use the shuttle *and*
the humans
33      together** ? and be ready to fold in more signals (audio, later pose) ? delivered
34      **MIT / Apache-2.0 / BSD-clean** so we can ramp commercially without a licensing
rethink.
35
36      This is **not greenfield** ? it operationalizes work the repo already blessed:
37
38      - **`HOW_RALLY_DETECTION_WORKS.md` open question:** *"Fusion: combine shuttle-motion
(WASB) with
39        player-motion for static cams."*
40      - **ADR-007** already designed a **modular "rally layer"** that fuses cues *outside* the
frozen WASB
41        model, with **audio as the #1 next cue and person/pose parked as #2**. This plan picks
up the parked
42        person cue and generalizes the fusion seam so future cues are "register a producer,"
not "re-touch
43        the segmenter."
44      - **`backend/pipeline/detectors/rally_gate.py` already exists** ? the net-crossing gate,
explicitly
45        labelled in its own docstring as *"Gate A in the over-fire fix spectrum (**B = player-
stance state
46        machine, future**)."* The person cue in this plan **is that named-but-unbuilt Gate
B.**
47
48      ### The owner's "held-shuttle" false positive ? what's actually going on
49
50      The owner observed a "rally" that was just a player **holding / flicking the shuttle in
hand** between
51      points. This is the *exact* failure `rally_gate.py` was written to kill:
52
53      > *"WASB over-detects: when a player flicks/tosses the shuttle back to the opponent for
service after
54      > losing a point, that single trajectory looks like a rally. The discriminator is
structural: a real
55      > rally has the shuttle crossing the net MULTIPLE times; a single service feed crosses
```

~once."*

The structural fix already exists and is validated offline ? but it is NOT wired into the production runtime path. `rally_gate.apply_gate(..., min_crossings=2)` is called only from the **eval drivers** (`distill_local.py`, `eval_partial_labels.py`, `export_wasb_segments.py`, `calibrate_local.py`); there is **no `min_crossings` knob in `IndexingConfig`** and the live segmenters don't apply it. So the single cheapest, highest-confidence win in this whole plan is **P2's first step: fold the existing Gate A into production.** Everything else (the person cue / Gate B, the learned fusion) is robustness on top of that.

Why humans help ? and the honest risk

The beachhead videos are **static tripod** (Bangalore academies, PMF_MARKET_RESEARCH.md), so player presence/motion is a usable cue. ADR-001's warning was specifically about **broadcast pan** cameras, where player-motion fooled the old heuristic ? *that failure mode does not apply to a fixed court cam.* But we keep **the shuttle as the camera-invariant primary** so the general bet stays robust on other camera types. Humans then add:

- **Precision** ? reject shuttle-on-floor / warm-up knock-ups / held-shuttle feeds when the court isn't occupied by ?2 players in a play posture.
- **Recall** (via the learned classifier) ? features that help bridge shuttle occlusion/blur gaps.

---

2. Core architecture ? one rally-fusion layer, cues are swappable producers

Formalize ADR-007's diagram. Every cue is a **black box that emits a time-aligned signal** (per-frame activity and/or per-window features); **fusion happens in *our* layer, never inside a frozen model.**

```
frames ?? WASB (frozen, MIT)      ?? shuttle trajectory ???
frames ?? person detector (ours)  ?? tracks + boxes ???????
audio  ?? hit detector (ours)     ?? hit events ???????????? RALLY LAYER (ours) ?? rallies
[frames ?? pose/stance (ours; parked #3) ?? stance ????????   feature + decision fusion
```

- **Shuttle = primary, camera-invariant.** Windows still **seed** from `TrackNetRunner.trajectory_to_action_windows()` (`tracknet_runner.py:234`). Other cues *enrich and adjudicate* shuttle-seeded windows; they do **not** replace the shuttle seed. Person runs only over **shuttle-seeded candidate windows + padding**, not the whole video ? which bounds the added cost.
- **A small `RallyCue` contract.** Each producer yields `(frame_idx ? features)` + per-window stats. Adding a cue = register a producer; the existing `SegmenterRegistry` (`segmenters/base.py:35`) is the template for the pattern. Keep the single-cue segmenters registered as fallbacks.

---

3. The two fusion modes (both ? per the owner decision)

3.1 Decision-level gate (safe baseline, ships first, no labels needed)

```
rally_active = shuttle_motion ? (?2 persons in court zone ? recent person track)
```

The `? recent-track` clause tolerates 1?2 frame person-detector misses so the AND-gate can't cause a
recall cliff. This composes with the **existing net-crossing Gate A**:

```
keep window  ?  net_crossings ? 2            (Gate A ? EXISTS in rally_gate.py, eval-only today)
                ?  (?2 in-court players ? recent track)   (Gate B ? the person cue, NEW)
```

Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-occupancy evidence
for the cases a single trajectory can still fool (e.g. a knock-up that does cross the net). No training
data required ? this is the label-free fusion that ships before any golden labels exist.

3.2 Feature-level (primary, the learned path)

The cues emit a **small set of aggregated, scale/translation-invariant per-window features ? NOT raw
coordinates.** Raw pixel coordinates would memorize *this* court/camera and fail to generalize off a
still-small labeled set; counts / ratios / zone-membership / two-sidedness transfer.

What already feeds the classifier (`distill_local.py:40`, `FEATURE_NAMES`):
`duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note **`net_crossings` is
already a feature** (computed via `rally_gate.gate_windows`).

Person features to add (~4, deliberately few):

Feature	Meaning	Rally signal
`players_in_court`	median count of persistent in-court tracks	? 2 (singles) / 4 (doubles); 0?1 = dead time
`two_sided_motion`	are moving players on **both** net-halves?	rally = two-sided; one person feeding = one-sided
`mean_player_motion`	aggregate normalized player velocity	rally = sustained; resting/standing = low
`in_court_ratio`	fraction of frames with ?2 in-court players	track stability vs flicker
`shuttle_player_colocation`	fraction of frames the shuttle sits inside/adjacent to a person box (hand region)	**held = high; in-flight = mostly free space** ? the direct held-shuttle discriminator

These join the shuttle features in the candidate `stats` JSON
(`database.py::add_candidate_segment`) and the feature vector of the **existing distilled classifier ?
the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004), NOT a new net.**
It scores each candidate window `P(real rally)` and replaces the Gemini call at runtime.

What the model learns: the **weights** on those few features ? e.g. down-weight a shuttle-motion
window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up, floor shuttle,

held-shuttle feed); up-weight ?2 players, two-sided, sustained motion, shuttle in free flight.

Small-N discipline (non-negotiable):
- Keep to **~4 person features** ? more features on a still-small labeled set overfit; that's exactly
  why the model stays a logistic regression, not a net.
- Use the **leave-one-video-out (LOVO)** protocol that `distill_local.py` **already implements**: train
  on the other videos' candidates, predict the held-out *video*, score vs its labels ? never a
  held-out *window* from the same video (windows in one video are correlated; that flatters the score).
- Training labels today are **Gemini silver labels** (distilled offline, ADR-004/007 ? Gemini is a
  teacher, never a runtime dependency, never a training-data *source* for owned weights); **golden
  labels are the honest measuring stick**, and as the Roora golden corpus grows they can also become
  training labels. Until the labeled set is larger, treat the learned weights as
  calibration / direction-check, not "a fully general model" ? the decision-level gate (§3.1) is
  the shippable, label-free fusion in the meantime.

> *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
> GOLDEN_SET_VENDORING.md.)*

---

4. Shuttle-state: in-flight vs in-hand (the misclassification, mapped to real code)

The "shuttle moved" seed conflates a shuttle **in flight** with a shuttle **carried in hand**. The
discriminators, and where each lives:

Signal	In-flight	Held / static	Status
`net_crossings`	? 1 (usually several)	~0 (held) / ~1 (single feed)	**EXISTS** (`rally_gate.py`); eval-only ? wire to prod
`shuttle_player_colocation`	low (free space)	high (near a hand)	**NEW** (needs the person cue)
`direction_reversals`	several (racket contacts)	~0	derivable from trajectory; partial overlap with crossings
`peak shuttle speed`	high	walking speed	derivable from existing velocity

So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation feature from the
person cue**. **No new label column is needed** ? the golden labels already start a rally at the
serve-contact frame and treat all pre-serve / held-shuttle time as dead time (see
ROORA_LABELING_GUIDELINES.md), so held-shuttle moments are negatives
by construction and are the ground truth that proves the detector wrong.

---

5. Adversarial mitigations (integrating safely)

- **Court-region mask** ? polygon around the court; drop persons outside it (spectators, coaches,
  adjacent courts). Config-supplied per camera; ~1 ms. Source: a manual per-camera polygon first;
  the **4 court corners + net-line** that Roora labels once per video (see the labeling doc) build
  this mask **and** the net split that `two_sided_motion` / `net_crossings` need.

- **Player-count constraint** ? expect **2 (singles) / 4 (doubles)** persistent in-court tracks;
  suppress transient/extra detections. The `singles_or_doubles` manifest flag sets the expectation.
- **Temporal association** ? reuse `supervision` **ByteTrack** (already in `yolo_hybrid`); reject
  singleton detections; smooth across frames.
- **Multi-court rejection** ? proximity to the primary court region (homography optional, later).
- **Domain shift (COCO frontal ? side/overhead court)** ? start **box-only** (robust); keep the
  detector swappable. Fine-tuning a person detector is **parked** ? we have no in-house person labels,
  and the **paid-LLM/training guardrail + minors-exclusion** apply to any future person-training data.
- **Fusion-degradation guard** ? shuttle stays primary; person is gate + feature, never the sole
  trigger; the single-cue segmenters remain registered fallbacks.

---

6. Box vs pose

Box-first. Occupancy + count + two-sided motion + shuttle-colocation is enough for both the gate
and the features, and box detection is the cheap, robust win. **2D pose is parked behind the same
`RallyCue` interface** ? pose (MediaPipe Pose / RTMPose, both Apache-2.0) buys serve-ready stance
gating and shot cues later, at the cost of a second model. This matches ADR-007 parking pose as the
#2/#3 cue.

---

7. Performance

The person detector is the **existing in-process torchvision model** run at **frame-skip 3?5**
(`yolo_frame_skip` default 3) with track interpolation between sampled frames. The shuttle keeps
running in its separate WSL process ? the two are **separate processes**, so a "shared backbone" isn't
free; budget realistically as the existing `yolo_hybrid` cost minus skip savings. Person runs **only
over shuttle-seeded windows + `ai_handoff_padding`**, not the whole video, which bounds the cost.

---

8. Licensing (the guardrail table ? all green)

Every dependency stays **MIT / Apache-2.0 / BSD** (ADR-002 / ADR-005; `ultralytics` was dropped for
AGPL-3.0).

Cue / model	Code	Weights / data	Verdict
Shuttle: WASB-SBDT, TrackNetV3	MIT	frozen badminton weights	? (ADR-001/002)
Person (default): torchvision Faster R-CNN / RetinaNet / FCOS	BSD/MIT	COCO-pretrained	? already in repo (`_DETECTOR_FACTORIES`)
Person (escalation): **YOLOX**, **RT-DETR / RT-DETRv2**	Apache-2.0	COCO / Objects365	?
Pose (parked): **MediaPipe Pose**, **RTMPose / MMPose**	Apache-2.0	COCO-structure	?

| Tracking: `supervision` ByteTrack | MIT | n/a | ? already in repo |
| **Banned** | **Ultralytics YOLOv5/8/11 (AGPL-3.0)** · **MonoTrack (Adobe, non-comm)** · **YOLO-NAS (Deci, non-comm)** | ? | ? |

COCO caveat to record: COCO *annotations* are **CC BY 4.0** (commercial OK *with attribution*);
COCO *images* are Flickr-ToS (per-image). Pretrained-**weight** use is clean; record the attribution.

---

9. Open decisions (for the owner)

1. **Court-region mask source** ? manual per-camera polygon (simple, ship first) vs auto court
   homography/line detection (later). **Recommend manual first**, fed by Roora's court-corner labels.
2. **Person detector default** ? keep torchvision (zero new deps, BSD/MIT) vs adopt YOLOX/RT-DETR
   (Apache, faster) now. **Recommend torchvision first**, escalate only if eval shows a speed/accuracy gap.
3. **Pose now or parked** ? **recommend parked** behind the interface (box-first).
4. **Golden labels** ? feature-level fusion (P3) is **blocked on golden rally labels**; the source is
   the VLC `rally-annotator` + the Roora vendor pilot (GOLDEN_SET_VENDORING.md).

---

10. Phased execution (owner-gated; ONE PR per slice, off `master`)

- **P0 ? this design doc** + ROORA_LABELING_GUIDELINES.md, indexed in
  `docs/README.md`, cross-linked from `HOW_RALLY_DETECTION_WORKS.md` and ADR-007. *(this deliverable;
  docs-only, no code.)*
- **P1 ? extract the shared person cue:** ? **DONE (2026-06-13).** Lifted the torchvision detector +
  ByteTrack + velocity out of `yolo_hybrid` into `backend/pipeline/detectors/person_detector.py`
  (`PersonDetector`); `yolo_hybrid` now orchestrates only and delegates to `self.person` (no behavior
  change ? regression-safe, proven by the unchanged process_video output tests). New
  `tests/test_person_detector.py`; suite +1 green; person_detector is type-clean (left mypy quarantine
  for the moved code). The fusion segmenter (P2) reuses this same producer over shuttle-seeded windows.
- **P2 ? wire Gate A into production, then add the court-mask + player-count gate (Gate B):** add a
  `min_crossings` knob + court-region polygon + count constraint to `IndexingConfig`; a
  `FusionSegmenter` (or extend the hybrid segmenter) runs the person cue over shuttle-seeded windows
  and applies `Gate A ? Gate B`. Persist person features into the candidate `stats`. **Wiring Gate A
  is the immediate false-positive win.**
- **P3 ? feature-level (learned) fusion:** add the person features (incl. `shuttle_player_colocation`)
  to `distill_local` / `classifier`; recalibrate IoU on the golden set; report **LOVO lift on held-out
  videos**. *(Gated on golden labels.)*
- **P4 ? additional cues:** wire **audio** (ADR-007's #1, already scaffolded in
  `backend/pipeline/audio/hit_detector.py`) and later **pose**, through the same `RallyCue` interface.
  Each is add-a-producer, not a refactor.

```
281     ---
282
283     ## 11. Verification
284
285     - **Offline, runnable in this dev container (no GPU/torch/cv2):** unit tests for the
fusion gate logic
286       and `RallyCue` aggregation (court-mask filter, player-count constraint, `? recent-
track` tolerance)
287       and the shuttle-state signals (net-crossing count ? extends the existing `rally_gate`
tests ? and
288       person-colocation) with **synthetic trajectories + person boxes**. A key-license
assertion test
289       (no `ultralytics` import; detector factories stay torchvision/Apache).
290     - **Machine-side (owner, GPU/WSL ? the full pipeline can't run in this container):** run
the
291       `FusionSegmenter` on a real static-tripod clip; confirm **held-shuttle / warm-up /
spectator false
292       positives drop** vs shuttle-only (this should reproduce the owner's observed bug and
show it fixed ?
293       start by confirming Gate-A-in-production alone fixes it); **LOVO IoU lift on the
golden set** for the
294       learned path.
295     - **Docs (P0):** links resolve; both docs indexed in `docs/README.md`; this doc cross-
linked from
296       `HOW_RALLY_DETECTION_WORKS.md` (the fusion open-question) and ADR-007.
297
```

6. assistant (2026-06-13T18:45:12.626Z)

Now let me read the relevant source files mentioned to understand the exact data structures and code:

7. user (2026-06-13T18:45:13.269Z)

```
1       """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3       Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4       the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5       `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6       next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7       later the learned classifier reads its persisted features. Keeping detection here (not
in
8       the segmenter) is what makes "add a cue = register a producer" true.
9
10      Behaviour is unchanged from the in-lined version ? only the home moved:
11        - `lazy_load_model`               ? load a permissively-licensed torchvision detector
12        - `make_tracker`                  ? a fresh supervision ByteTrack per chunk
13        - `detect_and_track`              ? one frame ? confident persons with persistent IDs
14        - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15          with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16          that feed the candidate `stats` JSON and the learned fusion features.
17
18      Heavy deps (torchvision, supervision) are imported lazily inside the methods so
importing
19      the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
20      the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
21      (ADR-005; ultralytics' AGPL YOLO was dropped).
22      """
23      import logging
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27      import torch
28
```

```
    from backend.utils.geometry import get_velocity_normalised

    logger = logging.getLogger(__name__)

    # torchvision detection models are trained on COCO where the "person" category is
    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
    _PERSON_CLASS_ID = 1

    # Permissively-licensed torchvision detectors selectable via config
    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
    _DETECTOR_FACTORIES = {
        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
    }
    _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"


    class PersonDetector:
        """Stage-1 person detection + tracking + motion-window extraction (a rally cue).

        Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
        ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
        min_action_window_duration}``); the model loads lazily on first use.
        """

        def __init__(self, config: Dict[str, Any]):
            self.config = config
            # Dynamic torchvision module (None until lazy_load_model); typed Any so the
            # `self.model([tensor])` inference call type-checks without a quarantine.
            self.model: Any = None
            self.device: Optional[str] = None

        def lazy_load_model(self) -> None:
            if self.model is not None:
                return

            # Imported lazily so the module imports without torchvision present and so
            # the test suite (which pre-sets self.model) never pulls in model weights.
            from torchvision.models import detection as tv_detection

            idx_cfg = self.config.get("indexing", {})
            use_gpu = idx_cfg.get("use_gpu", True)
            self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
            if use_gpu and not torch.cuda.is_available():
                logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")

            model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
            if model_name not in _DETECTOR_FACTORIES:
                logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
                model_name = _DEFAULT_DETECTOR

            builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
            logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
            # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
            self.model = builder(weights="DEFAULT")
            self.model.eval()
            self.model.to(self.device)

        def make_tracker(self, fps: float) -> Any:
            """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).

```

```
92              Imported lazily so tests can mock this method without supervision installed.
93              """
94              import supervision as sv
95              frame_rate = max(1, int(round(fps)))
96              return sv.ByteTrack(frame_rate=frame_rate)
97
98          def detect_and_track(self, frame: Any, tracker: Any,
99                               score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100             """Detect persons in a frame and assign persistent track IDs.
101
102             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103             This replaces the old ultralytics ``model.track(...)`` call, which bundled
104             detection AND association: torchvision does the detection, supervision's
105             ByteTrack does the association.
106             """
107             import supervision as sv
108
109             # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110             rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111             tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
112             with torch.no_grad():
113                 outputs = self.model([tensor])
114
115             out = outputs[0]
116             boxes = out["boxes"].detach().cpu().numpy()
117             labels = out["labels"].detach().cpu().numpy()
118             scores = out["scores"].detach().cpu().numpy()
119
120             # Keep only confident person detections.
121             mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122             if not mask.any():
123                 return []
124
125             # ByteTrack requires confidence to be populated (it filters on it internally).
126             dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127             tracked = tracker.update_with_detections(dets)
128
129             result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130             for i in range(len(tracked)):
131                 tid = tracked.tracker_id[i]
132                 if tid is None:
133                     continue
134                 x1, y1, x2, y2 = tracked.xyxy[i]
135                 result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136             return result
137
138         def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                               velocity_thresh: float, merge_gap: float,
140                                               frame_skip: int = 3,
141                                               chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142             """
143             Runs person detection + tracking on a video chunk and extracts high-motion
144             action windows. Returns a list of dicts in the full video's timeframe, each
with:
145                 start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146                 track_id_count, chunk_index.
147
148             Args:
149                 frame_skip: Process every Nth frame (1 = every frame). Higher values
150                             dramatically reduce inference cost with minimal quality loss.
151                 chunk_index: Index of the source chunk (recorded on each window for
```

```
traceability).
            """
            cap = cv2.VideoCapture(chunk_path)
            if not cap.isOpened():
                logger.error(f"Could not open {chunk_path}")
                return []

            fps = cap.get(cv2.CAP_PROP_FPS)
            if fps <= 0:
                fps = 30.0

            frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
            if frame_width <= 0:
                frame_width = 1280.0  # Sensible fallback

            # Effective sampling rate after frame-skipping
            effective_fps = fps / max(frame_skip, 1)

            # A fresh tracker per chunk ? track IDs only need to be consistent within a
            # chunk (windows are computed per chunk, then merged across chunks by time).
            tracker = self.make_tracker(effective_fps)
            score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)

            # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
            # so windows can be enriched with motion stats for logging + fine-tuning.
            active_frames = []
            frame_idx = 0
            track_history: Dict[int, Tuple[float, float, float, float]] = {}

            while True:
                ret, frame = cap.read()
                if not ret:
                    break

                # Skip frames for performance ? at 30fps, every 3rd frame still
                # gives ~10 samples/second, plenty for human-scale velocity.
                if frame_idx % max(frame_skip, 1) != 0:
                    frame_idx += 1
                    continue

                # Detect persons + assign persistent track IDs.
                detections = self.detect_and_track(frame, tracker, score_thresh)

                frame_peak_velocity = 0.0
                frame_active_ids = set()
                current_ids = set()

                for track_id, bbox in detections:
                    current_ids.add(track_id)

                    if track_id in track_history:
                        prev_bbox = track_history[track_id]
                        velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
                        if velocity > velocity_thresh:
                            frame_peak_velocity = max(frame_peak_velocity, velocity)
                            frame_active_ids.add(track_id)

                    track_history[track_id] = bbox

                # Prune stale IDs that are no longer visible ? prevents false
                # velocity spikes when the tracker reassigns an old ID.
                track_history = {k: v for k, v in track_history.items() if k in current_ids}

```

```
                if frame_active_ids:
                    active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))

                frame_idx += 1

            cap.release()

            # Group active frames into continuous windows
            if not active_frames:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
                velocities = [v for _, v, _ in group]
                track_ids = set().union(*[ids for _, _, ids in group])
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(velocities),
                    "mean_velocity": sum(velocities) / len(velocities),
                    "track_id_count": len(track_ids),
                    "chunk_index": chunk_index,
                }

            windows = []
            group = [active_frames[0]]
            for af in active_frames[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
                    group = [af]
            windows.append(_finalize(group))

            # Filter out extremely short action windows
            min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
            windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

            return windows

```

8. user (2026-06-13T18:45:13.504Z)

```
    """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).

    This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
    inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:

        1. Translate the Windows video path to its WSL-visible form (and optionally
           stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
        2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
        3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
        4. Convert that trajectory into high-motion "action windows" ? the same
           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
           pipeline (AI handoff, DB population) is unchanged.

    Nothing here imports TensorFlow; all model code stays in WSL.
    """

    import os
    import csv
    import shlex
```

```
    import logging
    import subprocess
    from dataclasses import dataclass
    from typing import List, Dict, Any, Optional, Tuple

    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote

    logger = logging.getLogger(__name__)


    @dataclass
    class TrackNetConfig:
        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
        distro: str = "Ubuntu"
        repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
        conda_env: str = "TrackNetV4"
        weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
        queue_length: int = 5
        stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
        wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
        timeout_sec: int = 1800                         # 30 min; kills a hung call (0 = no
timeout)
        keep_frames: bool = False                       # retain staged video after success
(debug only)
        min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
            tn = (idx_cfg or {}).get("tracknet", {}) or {}
            return cls(
                distro=tn.get("wsl_distro", "Ubuntu"),
                repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
                conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=tn.get("conda_env", "TrackNetV4"),
                weights_path=tn.get("weights_path", ""),
                queue_length=int(tn.get("queue_length", 5)),
                stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 1800)),
                keep_frames=bool(tn.get("keep_frames", False)),
                min_free_gb=float(tn.get("min_free_gb", 0.0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p
```

```

    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(WslCommandMixin, DetectorRunner):
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        def healthcheck(self) -> Tuple[bool, str]:
            """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

            Returns (ok, message). Cheap to call before a long run.
            """
            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"python -c \"import tensorflow as tf; "
                f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
            )
            try:
                res = self._wsl_bash(cmd)
            except FileNotFoundError:
                return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
            except subprocess.TimeoutExpired:
                return False, "WSL healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"WSL env check failed: {(res.stderr or out).strip()}"
            return True, out

        # ------------------------------------------------------------------ inference

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run predict.py on a video. Returns the Windows path to the produced CSV, or
None.

            ``output_win_dir`` is a Windows directory (under the repo's output/) that
            WSL writes into via its /mnt view, so the Windows process can read the CSV
            back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
            therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
            a filename cannot collide on the output CSV.
            """
            if not self.cfg.weights_path:
                logger.error(
                    "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
                    "in config.json to the WSL path of your trained TrackNetV4 weights."
                )
                return None

            # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
            # relative paths through unchanged, so WSL would resolve them against the
```

```
            # predict.py cwd instead of the repo (same bug class as wasb_runner).
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize separators so basename/splitext work when tests (or collector)
            # pass Windows-style paths on a Linux host.
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)

            # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
            if ext.lower() not in (".mp4", ".avi"):
                logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
                return None

            wsl_out = to_wsl_mnt_path(output_win_dir)
            # Namespace by video_id so two same-filename videos don't collide on the CSV.
            # predict.py derives its output name from the (staged) video stem, so staging
            # under the namespaced name propagates into "<key>_predict.csv".
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")

            # Resolve the video path WSL will read.
            if self.cfg.stage_in_wsl:
                staged = self._stage_video(video_win_path, f"{key}{ext}")
                if staged is None:
                    return None
                wsl_video = staged
            else:
                # No staging: predict.py reads /mnt directly and writes
"<stem>_predict.csv".
                # We cannot rename its output, so fall back to the un-namespaced name.
                wsl_video = to_wsl_mnt_path(video_win_path)
                expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")

            if self.cfg.min_free_gb > 0:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
                                   "consider running tools/cleanup_caches.py.",
                                   free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            cmd = (
                f"conda activate {self.cfg.conda_env} && "
                f"cd {self.cfg.repo_dir}/src && "
                f"python predict.py "
                f"--video_path {wsl_tilde_quote(wsl_video)} "
                f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--output_dir {wsl_tilde_quote(wsl_out)} "
                f"--queue_length {self.cfg.queue_length}"
            )
            logger.info("Running TrackNet inference on %s ...", base)
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
                return None

            if res.returncode != 0:
                logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
                return None

            if not os.path.exists(expected_csv_win):
                logger.error("TrackNet finished but expected CSV not found at %s",
```

```
expected_csv_win)
                return None
            logger.info("TrackNet trajectory CSV: %s", expected_csv_win)

            # Success ? the CSV is the durable output; drop the staged video copy (a pure,
            # regenerable intermediate). On earlier failure we return above without
cleaning.
            if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
                logger.info("Cache hygiene: removing staged video for %s "
                            "(set indexing.tracknet.keep_frames=true to retain).", key)
                self._wsl_rm_rf([wsl_video])
            return expected_csv_win

        def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
            """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).

            Returns the WSL path to the staged copy, or None on failure.
            """
            src_mnt = to_wsl_mnt_path(video_win_path)
            dest = f"{self.cfg.wsl_stage_dir}/{base}"
            cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
            try:
                res = self._wsl_bash(cmd)
            except subprocess.TimeoutExpired:
                logger.error("Staging video into WSL timed out.")
                return None
            if res.returncode != 0 or "OK" not in (res.stdout or ""):
                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
                return None
            return dest

        # ------------------------------------------------------------ CSV -> windows

        @staticmethod
        def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
            """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
            points: List[TrajectoryPoint] = []
            with open(csv_win_path, newline="") as f:
                reader = csv.DictReader(f)
                for row in reader:
                    try:
                        points.append(TrajectoryPoint(
                            frame=int(row["Frame"]),
                            visible=int(row["Visibility"]) == 1,
                            x=float(row["X"]),
                            y=float(row["Y"]),
                        ))
                    except (KeyError, ValueError):
                        continue
            points.sort(key=lambda p: p.frame)
            return points

        @staticmethod
        def trajectory_to_action_windows(
            points: List[TrajectoryPoint],
            fps: float,
            frame_width: float,
            chunk_start: float = 0.0,
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
```

```
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            # Compute per-frame normalised velocity from the last *visible* point.
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p

            if not active:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
```

9. user (2026-06-13T18:45:13.765Z)

```
"""Rally-start gate: suppress between-point false fires using shuttle net-crossings.

WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
the opponent for service after losing a point, that single trajectory looks like a
rally. The discriminator is structural, not duration-based: a real rally has the
shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
single service feed crosses ~once.

This module is camera-agnostic and needs NO new model ? it works on the trajectory
we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
axis with the larger shuttle spread) and the net position (median shuttle coord on
that axis ? the shuttle clusters at the two ends and crosses the net region often,
so the median sits near the net), then counts sign changes of (coord - net) across
the visible points inside a candidate window, with a deadband to ignore jitter.

Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
windows from trajectory_to_action_windows, or to fold into it later.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import List, Sequence, Tuple
```

```
    Interval = Tuple[float, float]


    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]
```

```
89
90
91      def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93          """Annotate each (start,end) window with its shuttle net-crossing count and a
94          keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95          axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96          axis, net, span = estimate_play_axis(points)
97          deadband = deadband_frac * span
98          out: List[GatedWindow] = []
99          for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                    kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113
```

10. assistant (2026-06-13T18:45:15.466Z)

Now let me read the classifier to see the existing feature structure and check the database for the stats JSON format:

11. user (2026-06-13T18:45:15.919Z)

```
1       """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3       The offline learned segmenter (ADR-004) must run locally with no external API and
4       no heavy ML stack ? the project already depends on numpy, and sklearn is not
5       installed. This implements just enough: standardized features, balanced-class
6       gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7       model is a small, human-readable, version-controllable file.
8
9       It deliberately stays simple: the rally-window features are few and roughly
10      linearly separable (high player motion + sustained activity => real rally), so a
11      well-regularized linear model is a sensible, inspectable baseline. Swap in a
12      richer model later behind the same fit/predict_proba/save/load interface.
13      """
14
15      import json
16      from typing import List, Optional
17
18      import numpy as np
19
20
21      class LogisticRegression:
22          def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23              self.lr = lr
24              self.epochs = epochs
25              self.l2 = l2
26              self.w: Optional[np.ndarray] = None
27              self.b: float = 0.0
28              self.mean: Optional[np.ndarray] = None
29              self.std: Optional[np.ndarray] = None
30              self.feature_names: Optional[List[str]] = None
31
```

```
        @staticmethod
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

        def _standardize(self, X: np.ndarray) -> np.ndarray:
            return (X - self.mean) / self.std

        def fit(self, X, y, feature_names: Optional[List[str]] = None,
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) ---------------------------

        def to_dict(self) -> dict:
            return {
                "model": "logreg",
                "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
                "w": self.w.tolist() if self.w is not None else None,
                "b": self.b,
                "mean": self.mean.tolist() if self.mean is not None else None,
                "std": self.std.tolist() if self.std is not None else None,
                "feature_names": self.feature_names,
            }

```

```
97          @classmethod
98          def from_dict(cls, d: dict) -> "LogisticRegression":
99              m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
```

12. user (2026-06-13T18:45:16.485Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer.
```

13. user (2026-06-13T18:45:17.224Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-35-from
backend.eval.metrics import score
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-36-from
backend.eval.gemini_refine import merge_overlaps
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-37-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-38-Interval =
Tuple[float, float]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-39-
backend\eval\distill_local.py:40:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity",
"active_ratio", "net_crossings"]
backend\eval\distill_local.py-41-# Recall-oriented proposal: deliberately permissive so real
rallies are among the
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-42-#
candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-43-PROPOSAL =
(0.005, 1.0, 0.5)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\eval\distill_local.py-44-BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's
production-ish windowing, no learning
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-45-
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-111-
clf, base = [], []
backend\eval\distill_local.py-112-        for i, held in enumerate(videos):
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-113-
others = [v for j, v in enumerate(videos) if j != i]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-114-
X = [row for v in others for row in v["X"]]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-115-
y = [t for v in others for t in v["y"]]
backend\eval\distill_local.py:116:            model = LogisticRegression().fit(X, y,
FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-117-
preds = predict_rallies(model, held["X"], held["intervals"], threshold)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-118-
sc = score(preds, held["gts"], iou)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-119-
bsc = score(held["baseline"], held["gts"], iou)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-120-
clf.append(sc)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-121-
base.append(bsc)
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-130-
result["mean_baseline_f1"] = mean(base, "f1")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-131-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-132-    #
Final model on ALL videos (for production / running on a fresh held-out video).
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-133-    Xall =
[row for v in videos for row in v["X"]]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-134-    yall =
[t for v in videos for t in v["y"]]
backend\eval\distill_local.py:135:    final = LogisticRegression().fit(Xall, yall,
```

```
FEATURE_NAMES)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-136-    assert
final.w is not None  # set by fit()
backend\eval\distill_local.py:137:    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for
w in final.w]))
backend\eval\distill_local.py-138-    print(f"\nfinal model standardized weights (sign\size =
influence): {weights}")
backend\eval\distill_local.py-139-    if model_out:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-140-
final.save(model_out)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-141-
print(f"final model (all videos) -> {model_out}")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py-142-
result["model_out"] = model_out
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-23-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-24-from
backend.eval.metrics import Interval, match_intervals
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-25-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-26-# YOLO Stage-1
persists these per candidate window in `candidate_segments.stats`
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-27-# (see
HybridYoloSegmenter). Duration is derived from the window bounds.
backend\eval\training.py:28:YOLO_FEATURE_NAMES = [
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-29-    "duration",
"peak_velocity", "mean_velocity", "active_frame_count", "track_id_count",
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-30-]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-31-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py-32-
backend\eval\training.py-33-def extract_yolo_features(row: Dict) -> List[float]:
```

14. assistant (2026-06-13T18:45:18.605Z)

Now let me look at the distill_local.py to understand how features are extracted and what the stats format is:

15. user (2026-06-13T18:45:18.845Z)

```
1       """Build a training set for the offline, API-free learned rally segmenter.
2
3       Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4       a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5       produces candidate rally windows with features; the collector's ground-truth
6       rally intervals tell us which windows are real. A lightweight classifier learns
7       that mapping, replacing the paid Gemini refinement with a local model.
8
9       This module is the *data layer* ? pure and unit-tested:
10
11        - `label_candidates`  ? generic: label windows positive/negative by whether they
12          match a ground-truth rally (same IoU matching the evaluator uses, so training
13          labels are consistent with how we score).
14        - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15          persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16        - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18      No model training or I/O here; that lives in the trainer/segmenter once the
19      substrate is chosen from the Phase 3 candidate-recall result.
20      """
21
22      from typing import Callable, Dict, List, Tuple
23
24      from backend.eval.metrics import Interval, match_intervals
25
26      # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27      # (see HybridYoloSegmenter). Duration is derived from the window bounds.
```

```
    YOLO_FEATURE_NAMES = [
        "duration", "peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count",
    ]


    def extract_yolo_features(row: Dict) -> List[float]:
        """Feature vector for one YOLO candidate row (dict from query_candidate_segments).

        `stats` is already JSON-deserialized by the DB layer. Missing fields default
        to 0.0 so a sparse/old row still yields a well-formed vector.
        """
        stats = row.get("stats") or {}
        duration = float(row["end_time"]) - float(row["start_time"])
        return [
            duration,
            float(stats.get("peak_velocity", 0.0)),
            float(stats.get("mean_velocity", 0.0)),
            float(stats.get("active_frame_count", 0)),
            float(stats.get("track_id_count", 0)),
        ]


    def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
                         iou_threshold: float = 0.5) -> List[int]:
        """Label each candidate window 1 (real rally) or 0 (spurious).

        A window is positive iff it is matched one-to-one to a ground-truth rally at
        `iou_threshold` ? the *same* matching the evaluator uses for true positives,
        so the classifier is trained on exactly what we score. Substrate-agnostic:
        works for YOLO or WASB-derived windows alike.
        """
        mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
        positive = {m.pred_index for m in mr.matches}
        return [1 if i in positive else 0 for i in range(len(candidate_intervals))]


    def build_training_set(
        candidate_rows: List[Dict],
        gt_intervals: List[Interval],
        iou_threshold: float = 0.5,
        feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
    ) -> Tuple[List[List[float]], List[int], List[Interval]]:
        """Turn one video's candidate rows + ground truth into (X, y, intervals).

        `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
        features without changing the labeling logic. Returns the feature matrix `X`,
        binary labels `y`, and the candidate intervals (handy for downstream merging).
        """
        intervals: List[Interval] = [
            (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
        ]
        y = label_candidates(intervals, gt_intervals, iou_threshold)
        X = [feature_fn(r) for r in candidate_rows]
        return X, y, intervals

```

16. user (2026-06-13T18:45:19.072Z)

```
    """Distill Gemini (teacher) into a LEARNED local rally classifier (student).

    Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
    further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
    using a few resolution/fps-INVARIANT local features, so the model transfers across
    videos and runs 100% local at inference (no Gemini, privacy lever intact).
```

```
7
8       Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10        filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11        active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12        -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Optional, Tuple
28
29
30      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31      from backend.pipeline.detectors import rally_gate
32      from backend.eval.calibrate_wasb import load_golden_gts
33      from backend.eval.training import label_candidates
34      from backend.eval.classifier import LogisticRegression
35      from backend.eval.metrics import score
36      from backend.eval.gemini_refine import merge_overlaps
37
38      Interval = Tuple[float, float]
39
40      FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41      # Recall-oriented proposal: deliberately permissive so real rallies are among the
42      # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
43      PROPOSAL = (0.005, 1.0, 0.5)
44      BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's production-ish windowing, no learning
45
46
47      def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
48                           proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
49          """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
50          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
51          vt, mg, mwd = proposal
52          wins = TrackNetRunner.trajectory_to_action_windows(
53              points, fps=fps, frame_width=frame_width,
54              velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
55          intervals = [(w["start"], w["end"]) for w in wins]
56          gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-
crossings
57          X: List[List[float]] = []
58          for w, g in zip(wins, gated):
59              dur = max(1e-6, w["end"] - w["start"])
60              win_frames = max(1.0, dur * fps)
61              X.append([
62                  dur,                                            # rally length (sec) ?
fps-invariant
63                  float(w["peak_velocity"]),                      # already normalized by
frame_width
64                  float(w["mean_velocity"]),
```

```
                float(w["active_frame_count"]) / win_frames,      # active-shuttle ratio ?
fps-invariant
                float(g.crossings),                               # net crossings (count) ?
the rally signal
            ])
        return intervals, X


    def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        vt, mg, mwd = BASELINE_LOCAL
        wins = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
        return [(w["start"], w["end"]) for w in wins]


    def load_video(spec: Dict, iou: float = 0.5) -> Dict:
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        intervals, X = build_candidates(spec["trajectory"], fps, fw)
        gts = load_golden_gts(spec["labels"])
        y = label_candidates(intervals, gts, iou)
        return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
                "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}


    def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
                        threshold: float = 0.5) -> List[Interval]:
        if not intervals:
            return []
        proba = model.predict_proba(X)
        pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
        return merge_overlaps(pos, gap_tol=1.0)


    def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
            model_out: Optional[str] = None) -> dict:
        videos = [load_video(s, iou) for s in specs]
        print(f"=== Distilled local rally classifier vs Gemini silver-GT "
              f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
        for v in videos:
            ceil = score(v["intervals"], v["gts"], iou).recall
            print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
                  f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

        result: dict = {"videos": [v["name"] for v in videos]}
        if len(videos) >= 2:
            print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
            clf, base = [], []
            for i, held in enumerate(videos):
                others = [v for j, v in enumerate(videos) if j != i]
                X = [row for v in others for row in v["X"]]
                y = [t for v in others for t in v["y"]]
                model = LogisticRegression().fit(X, y, FEATURE_NAMES)
                preds = predict_rallies(model, held["X"], held["intervals"], threshold)
                sc = score(preds, held["gts"], iou)
                bsc = score(held["baseline"], held["gts"], iou)
                clf.append(sc)
                base.append(bsc)
                print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
```

```
R={sc.recall:.3f} F1={sc.f1:.3f}"
123                       f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124             def mean(xs, a):
125                 return sum(getattr(s, a) for s in xs) / len(xs)
126             print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f}
R={mean(clf,'recall'):.3f} "
127                   f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
128                   f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
129             result["mean_learned_f1"] = mean(clf, "f1")
130             result["mean_baseline_f1"] = mean(base, "f1")
131
132         # Final model on ALL videos (for production / running on a fresh held-out video).
133         Xall = [row for v in videos for row in v["X"]]
134         yall = [t for v in videos for t in v["y"]]
135         final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136         assert final.w is not None  # set by fit()
137         weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138         print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139         if model_out:
140             final.save(model_out)
141             print(f"final model (all videos) -> {model_out}")
142             result["model_out"] = model_out
143         return result
144
145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,
149                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161
```

17. assistant (2026-06-13T18:45:20.485Z)

Now let me check the segmenters to understand how stats are persisted into the database:

18. user (2026-06-13T18:45:21.290Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/segmenters/base.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/segmenters/gemini.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/segmenters/motion.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/backend/pipeline/segmenters/tracknet_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/backend/pipeline/segmenters/trajectory_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/segmenters/wasb_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/segmenters/yolo_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/pipeline/segmenters/__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer/scratch/test_ai_segmenter.py
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_database.py
```

19. user (2026-06-13T18:45:23.206Z)

```
85-                )
86-            """)
87-
88-            # Versioned migrations live here. PRAGMA user_version is the schema
89-            # contract ? bump it for every change and add an ordered `if version < N`
90-            # block below. Tests assert the expected version, so accidental drift fails CI.
91-            version = cursor.execute("PRAGMA user_version").fetchone()[0]
92-
93-            if version < 1:
94-                # v1: raw candidate detections (pre-confirmation), detector-agnostic.
95:                # Common fields are columns; detector-specific metrics go in `stats` JSON,
96:                # so a new segmenter reuses this table by setting its own `detector`/`stats`.
97-                cursor.execute("""
98-                    CREATE TABLE IF NOT EXISTS candidate_segments (
99-                        id INTEGER PRIMARY KEY AUTOINCREMENT,
100-                         video_id TEXT NOT NULL,
101-                         detector TEXT NOT NULL,
102-                         chunk_index INTEGER,
103-                         start_time REAL NOT NULL,
104-                         end_time REAL NOT NULL,
105-                         duration REAL NOT NULL,
106-                         confidence REAL,
107:                         stats TEXT,
108-                         detection_params TEXT,
109-                         confirmed_segment_id INTEGER,
110-                         human_label TEXT,
111-                         notes TEXT,
112-                         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113-                         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114-                         FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
115-                     )
116-                """)
117-                cursor.execute("""
--
230-                conn.cursor().execute(
231-                    "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
232-                    (segment_id, timestamp, event_type, player, json.dumps(details or {}))
233-                )
234-                conn.commit()
235-            return True
236-        except Exception as e:
237-            logger.error(f"Failed to add event: {e}")
238-            return False
239-
240:    def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
241-                              chunk_index: Optional[int] = None, confidence: Optional[float]
= None,
242:                              stats: Optional[Dict[str, Any]] = None,
243-                              detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
244:        """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
245-        detector-specific dicts stored as JSON. Returns the new candidate id, or None."""
246-        try:
247-            with self._connect() as conn:
248-                cursor = conn.cursor()
249-                cursor.execute(
250-                    """INSERT INTO candidate_segments
251-                       (video_id, detector, chunk_index, start_time, end_time, duration,
252:                        confidence, stats, detection_params)
253-                       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
```

```
254-                    (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
255:                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
256-                )
257-                conn.commit()
258-                return cursor.lastrowid
259-        except Exception as e:
260-            logger.error(f"Failed to add candidate segment: {e}")
261-            return None
262-
263-    def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
264-        """Record that a candidate detection was confirmed as a given play_segment."""
265-        try:
--
270-                )
271-                conn.commit()
272-            return True
273-        except Exception as e:
274-            logger.error(f"Failed to link candidate {candidate_id} to segment {segment_id}:
{e}")
275-            return False
276-
277-    def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
278-                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
279-        """Fetch candidate detections for the annotation / fine-tuning workflow.
280:        `stats` and `detection_params` are deserialized from JSON."""
281-        query = "SELECT * FROM candidate_segments WHERE video_id = ?"
282-        params: List[Any] = [video_id]
283-
284-        if detector:
285-            query += " AND detector = ?"
286-            params.append(detector)
287-        if only_unlabeled:
288-            query += " AND human_label IS NULL"
289-
290-        query += " ORDER BY start_time"
291-
292-        candidates = []
293-        with self._connect() as conn:
294-            rows = conn.cursor().execute(query, params).fetchall()
295-            for row in rows:
296-                d = dict(row)
297:                d["stats"] = json.loads(d["stats"] or "{}")
298-                d["detection_params"] = json.loads(d["detection_params"] or "{}")
```

[Showing results with pagination = limit: 100]

20. assistant (2026-06-13T18:45:27.533Z)

Perfect. Now let me check the yolo_hybrid segmenter to see exactly how it persists stats:

21. user (2026-06-13T18:45:28.118Z)

```
1       """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2       (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4       Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5       (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6       can be consumed by the future fusion segmenter; this segmenter just orchestrates
7       chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9       Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10      service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11      detection uses torchvision's permissively-licensed (BSD) detectors and the
```

```
    association/tracking that YOLO.track() used to bundle in is provided by
    supervision's MIT-licensed ByteTrack.

    The registry key stays "yolo_hybrid" for back-compat with existing configs and the
    DB `detector` tags, even though no YOLO is involved anymore.
    """
    import os
    import time
    import logging
    import concurrent.futures
    from typing import List, Dict, Any, Tuple

    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
    from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class HybridYoloSegmenter(BasePlaySegmenter):
        def __init__(self, db, config):
            super().__init__(db, config)
            # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
producer
            # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
            self.person = PersonDetector(config)

        @property
        def name(self) -> str:
            return "yolo_hybrid"

        @property
        def description(self) -> str:
            return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
                    "candidate filtering, then an AI provider for high-precision semantic
boundaries.")

        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
            self.person.lazy_load_model()

            # Get sport profile config
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            # Get AI provider and model config
            idx_cfg = self.config.get("indexing", {})
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
```

```
72
73              # Get appropriate API key based on the provider
74              api_key_env_var = f"{provider_name.upper()}_API_KEY"
75              api_key = os.environ.get(api_key_env_var)
76              if not api_key:
77                  logger.error(
78                      f"{api_key_env_var} environment variable is not set! "
79                      f"To run the {provider_name} segmenter, set your API key. "
80                      f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
81                  )
82                  return []
83
84              try:
85                  provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
86              except KeyError as e:
87                  logger.error(str(e))
88                  return []
89
90              logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
91
92              video_duration = get_video_duration(video_path)
93              if video_duration <= 0:
94                  logger.error("Invalid video duration.")
95                  return []
96
97              chunk_duration = idx_cfg.get("chunk_duration", 300)
98              chunk_overlap = idx_cfg.get("chunk_overlap", 30)
99              velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15)  # Fraction of
frame width per second
100             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
101             frame_skip = idx_cfg.get("yolo_frame_skip", 3)
102             tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
103             handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
104             ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
105             log_candidates = idx_cfg.get("log_yolo_candidates", True)
106             store_candidates = idx_cfg.get("store_yolo_candidates", True)
107
108             # Snapshot of the params that produced these detections ? stored alongside each
109             # candidate so a fine-tuning run can reproduce / correlate against them.
110             detection_params = {
111                 "velocity_thresh": velocity_thresh,
112                 "merge_gap": merge_gap,
113                 "frame_skip": frame_skip,
114                 "tracker": tracker_config,
115                 "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
116                 "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
117                 "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
118             }
119
120             chunks_dir = os.path.join("output", "chunks_yolo")
121             os.makedirs(chunks_dir, exist_ok=True)
122
123             step = chunk_duration - chunk_overlap
124             chunk_starts = []
125             t = 0.0
126             while t < video_duration:
127                 chunk_starts.append(t)
128                 t += step
129
130             num_chunks = len(chunk_starts)
```

```
            logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")

            all_candidate_windows = []

            t_start = time.time()
            prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)

            if num_chunks > 1 and prefetch_workers > 0:
                # --- PIPELINED PROCESSING ---
                # GPU inference must stay sequential (single GPU), but ffmpeg chunk
                # extraction is pure I/O. We pre-extract upcoming chunks in background
                # threads so extraction of chunk N+1 overlaps with inference on chunk N.
                logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")

                extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)

                def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
                    chunk_end = min(cs + chunk_duration, video_duration)
                    actual_dur = chunk_end - cs
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
                    success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
                    return (ci, cs, chunk_path, success)

                # Submit all extractions upfront ? they'll queue and run as workers free up
                extract_futures = [
                    extract_executor.submit(_extract_chunk, ci, cs)
                    for ci, cs in enumerate(chunk_starts)
                ]

                # Process results in order: wait for extraction, then run inference on GPU
                for future in extract_futures:
                    ci, chunk_start, chunk_path, success = future.result()
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")

                    if not success:
                        logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
                        chunk_path, chunk_start, velocity_thresh, merge_gap,
                        frame_skip=frame_skip, chunk_index=ci
                    )
                    logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                extract_executor.shutdown(wait=False)
            else:
                # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
                for ci, chunk_start in enumerate(chunk_starts):
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_dur = chunk_end - chunk_start
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
```

```
{chunk_end:.1f}s)...")
                    success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
                    if not success:
                        continue

                    windows = self.person.extract_action_windows_from_chunk(
                        chunk_path, chunk_start, velocity_thresh, merge_gap,
                        frame_skip=frame_skip, chunk_index=ci
                    )
                    logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
                    all_candidate_windows.extend(windows)

                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

            # Deduplicate overlapping candidate windows across chunks (chunks overlap by
            # chunk_overlap, so the same rally can surface in two adjacent chunks).
            def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
                fa, fb = a["active_frame_count"], b["active_frame_count"]
                total = fa + fb or 1
                return {
                    "start": a["start"],
                    "end": max(a["end"], b["end"]),
                    "active_frame_count": fa + fb,
                    "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
                    # Frame-weighted mean so longer sub-windows dominate proportionally.
                    "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
                    "track_id_count": max(a["track_id_count"], b["track_id_count"]),
                    "chunk_index": a["chunk_index"],
                }

            if all_candidate_windows:
                all_candidate_windows.sort(key=lambda w: w["start"])
                merged_windows = [all_candidate_windows[0]]
                for current in all_candidate_windows[1:]:
                    last = merged_windows[-1]
                    if current["start"] <= last["end"]:  # Overlap
                        merged_windows[-1] = _merge_stats(last, current)
                    else:
                        merged_windows.append(current)
                all_candidate_windows = merged_windows

            logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
            logger.info(f"Total candidate windows: {len(all_candidate_windows)}")

            # Per-rally console log (final, full-video timestamps) for video cross-
verification.
            if log_candidates:
                for w in all_candidate_windows:
                    logger.info(
                        f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
                        f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}
"
                        f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
                        f"tracks={w['track_id_count']}"
                    )

            # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
            # so confirmed AI segments can be linked back in Phase 4.
```

```
249             candidate_ids = [None] * len(all_candidate_windows)
250             if store_candidates:
251                 for i, w in enumerate(all_candidate_windows):
252                     stats = {
253                         "peak_velocity": w["peak_velocity"],
254                         "mean_velocity": w["mean_velocity"],
255                         "active_frame_count": w["active_frame_count"],
256                         "track_id_count": w["track_id_count"],
257                     }
258                     # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
259                     confidence = min(1.0, w["peak_velocity"])
260                     candidate_ids[i] = self.db.add_candidate_segment(
261                         video_id, detector="yolo_hybrid",
262                         start_time=w["start"], end_time=w["end"],
263                         chunk_index=w["chunk_index"], confidence=confidence,
264                         stats=stats, detection_params=detection_params,
265                     )
266
267             try:
268                 os.rmdir(chunks_dir)
269             except Exception:
270                 pass
271
272             # Phase 3: AI Provider Handoff
273             gemini_segments = []
274             gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
275             os.makedirs(gemini_dir, exist_ok=True)
276
277             logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
278
279             def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
280                 w_start = max(0, window["start"] - handoff_padding)
281                 w_end = min(video_duration, window["end"] + handoff_padding)
282                 w_dur = w_end - w_start
283
284                 w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
285                 # Re-encode short handoff clips to avoid keyframe-boundary drift
286                 success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
287                 if not success:
288                     return []
289
290                 prompt = sport_profile.get_prompt(chunk_duration=w_dur)
291                 segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
292
293                 # Coordinate mapping ? shift local timestamps back to full-video time
294                 valid_segments = []
295                 for segment in segments:
296                     try:
297                         segment["start_time"] = float(segment["start_time"]) + w_start
298                         segment["end_time"] = float(segment["end_time"]) + w_start
299                         # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
300                         segment["_candidate_id"] = candidate_ids[wi]
```

22. assistant (2026-06-13T18:45:30.308Z)

Excellent. Now I have a clear picture. Let me create a comprehensive design document for the person-feature computation as pure functions. Let me also check the geometry utilities to understand how to normalize coordinates:

23. user (2026-06-13T18:45:30.847Z)

```
    import math
    from typing import Tuple

    def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
        """Calculate the center (x, y) of a bounding box.
        bbox format: (x1, y1, x2, y2)
        """
        x1, y1, x2, y2 = bbox
        return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)

    def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
        """Calculate the Euclidean distance between two points (x, y)."""
        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)

    def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
        """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
        """
        c1 = get_center(bbox1)
        c2 = get_center(bbox2)
        dist = get_distance(c1, c2)
        return dist * fps

    def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
                                fps: float, frame_width: float) -> float:
        """Calculate velocity as a fraction of frame width per second.

        This makes the threshold resolution-independent:
        - 0.15 means the center moved 15% of the frame width in one second.
        - Works identically on 720p, 1080p, and 4K footage.

        Args:
            bbox1: Previous bounding box (x1, y1, x2, y2).
            bbox2: Current bounding box (x1, y1, x2, y2).
            fps: Effective frames per second (after frame-skipping).
            frame_width: Width of the video frame in pixels.
        """
        raw_velocity = get_velocity(bbox1, bbox2, fps)
        if frame_width <= 0:
            return raw_velocity  # Fallback to raw pixels if width unknown
        return raw_velocity / frame_width

    def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
        """Calculate the Intersection over Union (IoU) of two bounding boxes.
        bbox format: (x1, y1, x2, y2)
        """
        xA = max(boxA[0], boxB[0])
        yA = max(boxA[1], boxB[1])
        xB = min(boxA[2], boxB[2])
        yB = min(boxA[3], boxB[3])

        interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
        if interArea == 0:
            return 0.0

        boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
        boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)

        iou = interArea / float(boxAArea + boxBArea - interArea)
        return iou

```

24. assistant (2026-06-13T18:45:32.433Z)

Perfect. Now I have all the context I need. Let me call StructuredOutput with my findings, which will be a design specification for the pure-function person-feature computation:

25. user (2026-06-13T18:46:20.239Z)

Structured output provided successfully

26. assistant (2026-06-13T18:46:26.212Z)

Perfect! I've completed a comprehensive pure-function design specification for the person-feature computation. Here's a summary of what I've delivered:

Summary

I've designed **five scale/translation-invariant person-cue features** as pure functions, grounded in the exact codebase:

Features Designed:

1. **`players_in_court`** ? Median count of in-court persons per frame (counts / ratios, never pixels)
2. **`two_sided_motion`** ? Fraction of frames with active motion on both net-halves (detects one-sided feeds vs. rallies)
3. **`mean_player_motion`** ? Aggregate normalized player velocity (already handled by PersonDetector; just aggregates)
4. **`in_court_ratio`** ? Fraction of frames with ?2 stable in-court players (detects tracking flicker vs. real occupancy)
5. **`shuttle_player_colocation`** ? Fraction of visible-shuttle frames overlapping person boxes (direct held-shuttle discriminator ? high overlap = held, low = in-flight)

Key Design Properties:

- **Pure functions only** ? no I/O, no models, no state
- **Edge cases covered**: No shuttle visible, <2 persons, polygon=None, net line missing
- **Helper formulas included**:
  - `point_in_polygon` (ray-casting, normalized 0?1)
  - `normalize_bbox_to_court` (pixel?normalized)
  - `compute_in_court_per_frame` (per-frame court filtering)
- **Integration points explicit**: How each feeds distill_local.FEATURE_NAMES and candidate_segments.stats JSON
- **Risks documented**: Visibility flag assumptions, active-motion filtering requirements, polygon validation

All formulas are quoted from actual file:line anchors in the indexer repo. Ready for P2/P3 implementation.